

Adopting timezone-aware UTC as the datetime foundation

The framework will store, reason about, and serve every timestamp as a true UTC instant. This document explains exactly what changes — in the database, in the code, and in the frontend — proves it with reproducible evidence, and shows why it beats every alternative we considered.

Decision `USE_TZ = True` · `TIME_ZONE = "UTC"` **Status** Accepted

Evidence live DB + DRF, 2026-07-20

THE DECISION

Set `USE_TZ = True` and `TIME_ZONE = "UTC"`.

Datetimes become timezone-aware UTC instants everywhere: the ORM returns aware objects, the API serves an explicit offset, and each client renders in the viewer's own zone. Values that represent a *calendar day* rather than a *moment* become `DateField`. This is the only configuration in which a user in Berlin, New York, or Tokyo all see the correct time — and we prove that below.

01 PROOF FIRST — WHAT THE TWO SETTINGS ACTUALLY DO

The entire decision rests on one measured fact, so we lead with it. Below is the real behaviour of Django REST Framework's `DateTimeField` — the exact code every write passes through — for three kinds of input, under the old setting and the new one. Same machine, same field, only `USE_TZ` flipped:

```

==== USE_TZ=False, TIME_ZONE="UTC" (today) ====
in: 2026-07-20T11:00:00      -> 2026-07-20 11:00:00      (naive)
in: 2026-07-20T11:00:00+02:00 -> 2026-07-20 09:00:00      (naive)
in: 2026-07-20T11:00:00Z      -> 2026-07-20 11:00:00      (naive)

==== USE_TZ=True, TIME_ZONE="UTC" (decision) ====
in: 2026-07-20T11:00:00      -> 2026-07-20 11:00:00+00:00 (AWARE)
in: 2026-07-20T11:00:00+02:00 -> 2026-07-20 09:00:00+00:00 (AWARE)
in: 2026-07-20T11:00:00Z      -> 2026-07-20 11:00:00+00:00 (AWARE)

```

Three things are proven at once, and they shape everything that follows:

- **The stored instant is identical** in both modes (11:00 → 11:00, +02:00 → 09:00, Z → 11:00). The columns already hold UTC — so **flipping to `USE_TZ=True` needs no data migration**.
- **The type changes: naive → aware**. Under the new setting every value carries `+00:00` — the "this is UTC" fact lives *in the value*, not in a fragile serializer trick.
- **An explicit offset is honoured** (`11:00+02:00` → `09:00Z`) in both modes. This is the crux: correctness comes from the *client sending the offset*, which is why the frontend change (§4) is not optional.

The honest caveat, stated up front

Look at the first row: a *naive* `11:00` still stores as `11:00Z` under the new setting — a Berlin user would still see 13:00. **The flip alone does not fix the bug**. It fixes it *together with* the frontend sending real instants. What the flip changes on its own is that the mistake stops being silent — see §3.

02 WHAT CHANGES IN THE DATABASE

Storage layout: nothing changes. Django's PostgreSQL backend already creates every `DateTimeField` as `timestamp with time zone` (`timestampz`) — even under `USE_TZ=False`. Verified on the live database:

```
● ● ● information_schema.columns - live
```

```
created_at    -> timestamp with time zone
edited_at     -> timestamp with time zone
valid_from    -> timestamp with time zone
sys_from      -> timestamp with time zone
```

A `timestampz` column never stores a timezone — it stores an **absolute instant**. The session timezone only decides how a bare value is *interpreted on the way in* and *displayed on the way out*. We confirmed the interpretation rule directly: with the session zone set, a naive `11:00` written into a `timestampz` comes back as the instant it was anchored to. That means:

- **No `ALTER COLUMN`, no type change, no table rewrite.** The flip is a settings change at the storage layer.
- **Existing rows keep their instant.** Reading an old value under `USE_TZ=True` returns the same moment, now as an aware UTC datetime.
- **The connection is pinned to UTC** (`TIME_ZONE="UTC"`), so bare/legacy values are read as UTC — which is what they already are.

What about already-wrong rows?

A small window of *user-entered* dates written after the rc212 upgrade were anchored to the wrong zone and are off by an offset. Those are the only corrupted rows; app-stamped fields (`created_at` , `history`, `audit`) were always correct. Whether you rebase them or let them self-heal on next edit is the one open choice — covered in the companion “*one destination, two ways*” brief. It does not change this decision.

03 WHAT CHANGES IN THE CODE

This is where the real work is. Every datetime the framework hands you becomes **aware**. That is a strict upgrade in correctness, but it has consequences any Python code touching datetimes must respect.

The clock helpers already do the right thing

```
def lex_datetime_now():
    if settings.USE_TZ:
        return timezone.now()          # aware UTC – the new path
    return datetime.now(...).replace(tzinfo=None) # naive – retired
```

`lex_datetime_now()`, `timezone.now()`, `auto_now` / `auto_now_add`, and the history stamps (`valid_from`, `sys_from`) all return aware UTC the moment the flag flips. History, audit, and calculation timestamps become correct-by-construction with zero edits.

Naive datetimes become loud — the safety win

Under the old setting, a naive value slid through silently and was *assumed* to be UTC. Under the new one, persisting a naive datetime warns — proven:

```
... DateTimeField.get_prep_value(datetime(2026,7,20,11,0,0)) under
USE_TZ=True

result : 2026-07-20 11:00:00+00:00    (AWARE)
warned : True
-> RuntimeWarning: DateTimeField received a naive datetime
    (2026-07-20 11:00:00) while time zone support is active.
```

We turn that warning into a **test error**, so a stray naive datetime fails CI at the exact line instead of shipping a silent one-offset bug.

What breaks, and the fix

✗ BREAKS UNDER USE_TZ=TRUE

Comparing a stored (now aware) field to a stdlib naive value:

```
obj.created_at < datetime.now()
→ TypeError: can't compare offset-
naive and offset-aware datetimes
```

✓ CORRECT

Use the framework clock — always aware:

```
obj.created_at <
lex_datetime_now()
```

The crash is a *feature*: it points at the exact line that was silently drifting before.

✗ A CALENDAR DAY MODELLED AS A MOMENT

```
output_date = datetime(year, 12, 31) on a DateTimeField
```

→ becomes an instant; a US viewer sees **Dec 30**.

✓ MODEL IT AS A DAY

```
output_date = models.DateField(),  
set with date(year, 12, 31)
```

→ Dec 31 for everyone, no zone math possible.

The sweep is mechanical and enforced: replace stray `datetime.now()` with `lex_datetime_now()`, and add ruff's `DTZ` rules so naive `datetime.now()` / `datetime(...)` can't merge again. Reads of existing data need *no* change — old rows return as correct aware UTC instants.

04 HOW THE FRONTEND ADAPTS

This is the half that actually closes the visible bug. Today the date picker sends a **naive local wall-clock** string with no zone — which §1 proved gets read as UTC. The fix is two lines, one on each leg.

Write — stamp the user's zone at the one place it's known

```
src/utils - dateParser (react-admin DateTimeInput)  
  
// before: passes the naive wall-clock straight through  
export const dateParser = (v) => v // "2026-07-20T11:00"  
  
// after: turn the local pick into an explicit instant  
export const dateParser = (v) => v ? new Date(v).toISOString() : v  
// "2026-07-20T09:00:00.000Z"
```

`new Date("2026-07-20T11:00")` interprets the picked value in the *browser's* zone — exactly the fact we were throwing away — and `toISOString()` emits the true UTC instant. A New York user's 11:00 becomes 15:00Z; a Berlin user's becomes 09:00Z. The server no longer has to guess.

Read — the existing code becomes correct

date/time cell + field rendering

```
// unchanged – but now the value carries a real 'Z', so this is finally right
new Date(value).toLocaleString() // renders in the viewer's local zone
```

Date-only fields must NOT go through `new Date()`

A `DateField` value is a plain `"2026-12-31"` string with no time and no zone. Keep it that way end to end — never wrap it in `new Date()` / `toISOString()`, or you reintroduce a midnight that can shift a day. react-admin's `DateInput` already handles this; the rule is simply "don't zone-convert a day."

One more thing to align: AG-Grid date **filters** should compare instants, so a filter built from a picked local date is converted to an instant the same way before it hits the API. Same principle, same one-line helper.

05 THE WHOLE THING, WORKED END TO END

Anne is in Berlin (summer, UTC+2). She picks **11:00** in a datetime field. Follow the value through the fixed system, and watch what three different people see:

STEP	VALUE	NOTE
Anne picks	11:00 Berlin	her local wall-clock
Frontend sends	2026-07-20T09:00:00.000Z	toISOString captures +02:00
DRF stores	09:00:00+00:00	aware UTC instant
DB column	09:00Z in timestamptz	one absolute moment
API serves	2026-07-20T09:00:00Z	explicit, honest offset

WHO READS IT	SEES	CORRECT?
Anne — Berlin (UTC+2)	11:00	✓ what she entered
Bob — New York (UTC-4)	05:00	✓ same moment, his clock
Consultant — Tokyo, DB copy on a local laptop	18:00	✓ machine zone can't corrupt it
A calculation comparing two timestamps	true instants	✓ always
output_date (a DateField)	Dec 31	✓ same day everywhere

06 WHAT DEVELOPERS & USERS KEEP IN MIND

- **Moment vs day is a modelling decision.** "When did it happen" → `DateTimeField`. "Which calendar day" → `DateField`. Most timezone pain is a day wearing a moment's clothes.
- **Never build a naive datetime in app code.** Use `lex_datetime_now()` / `timezone.now()`; for a date use `date(y, m, d)`, not `datetime(y, m, d)`. CI enforces this.
- **Aware values compare cleanly with each other and crash against naive ones** — on purpose. A `TypeError` is the system pointing at a latent bug, not an obstacle.
- **Downloading the production DB to a laptop is now safe.** Instants are absolute; your machine's timezone changes only what you see, never what's stored.
- **End users just see their own clock.** No setting to configure, no "server time" to mentally convert. The moment is the same for everyone; the rendering is theirs.

07 WHY THIS BEATS THE ALTERNATIVES

We considered every configuration reachable from here. Each fails the one requirement — *any user, any region, correct instant* — except this one.

APPROACH	WHAT IT DOES	WHY IT LOSES
Today: <code>False + UTC + Z-hack</code>	Naive UTC storage; a serializer format appends a Z.	Can't honour a viewer's zone; the Z is a hack that lies the moment a naive-local value arrives. Round-trip drifts by the offset.
Floating: <code>False + Berlin + no Z</code>	Everyone sees the same digits; no zone math at all.	Fine for a single office; wrong for real instants across zones — a "moment" means a different time to each viewer. Calculations compare wall-clocks.
Anchor-to-Berlin serializer	Labels naive values with the business zone's offset.	Correct for Berlin instants, but re-breaks calendar dates for a US viewer and hard-codes one office as the centre of the world.
This: <code>True + UTC</code>	Aware UTC instants; client sends & renders offsets.	Correct for every region; UTC by type, not by hope; naive mistakes are loud; no column migration.

The clinching proof

Two regression tests encode the two things a client needs: **12g** — every served datetime carries a timezone designator; **12j** — a value entered as X reads back as X. We ran both under each setting. Under *any* `USE_TZ=False` config, exactly one is green and the other red — they are mutually exclusive. **Only `USE_TZ=True` makes both green at once.** That isn't a preference; it's a proof that this is the only configuration where correctness is representable.

08 WHAT SHIPPING IT LOOKS LIKE

- **Backend:** `USE_TZ=True`, `TIME_ZONE="UTC"`, delete the BUG-025 Z hack, sweep the four stray `datetime.now()` call sites → `lex_datetime_now()`.
- **Frontend:** `dateParser` → `toISOString()` on write; leave the local-render on read; keep `DateField` values as plain date strings.

- **Schema, rolling:** convert civil `DateTimeField`s (`output_date` , effective dates) → `DateField` per project as touched.
- **Guardrails:** naive-datetime warning → test error; ruff `DTZ` in CI; 12g + 12j green together as the standing gate.
- **Data:** no column migration ever. The only open choice is whether to rebase the small window of post-rc212 user dates or let them self-heal — see the companion brief.

lex-app timezone architecture decision · every figure and terminal capture in this document was produced against the live test database, DRF, and git history on 2026-07-20 and is reproducible. Companion: [One destination, two ways — the data-safe vs data-unsafe brief](#).